

# Relazione - Versione 4: Planner

## Introduzione

La Versione 4 introduce il Planner.

Nella versione precedente, il progetto ha introdotto il Generic Parser. L'agente era già in grado di selezionare uno strumento, estrarre gli argomenti richiesti, eseguire lo strumento selezionato e memorizzare l'interazione.

La Versione 4 mantiene questa struttura, ma aggiunge un nuovo passaggio prima del parsing e dell'esecuzione: ora l'agente crea un semplice piano di esecuzione.

Questa versione è ancora basata su regole. Non viene utilizzato alcun LLM e non viene introdotto alcun framework esterno.

## Obiettivo della Versione 4

L'obiettivo principale della Versione 4 è introdurre la pianificazione.

L'agente non deve eseguire immediatamente un'azione dopo aver selezionato uno strumento. Deve prima creare un piccolo piano che descriva ciò che sta per fare.

Il piano contiene tre informazioni:

- lo strumento selezionato;
- il motivo per cui viene utilizzato;
- gli argomenti richiesti dallo strumento.

L'idea importante è semplice: l'agente inizia a pianificare prima di agire.

## Cosa cambia rispetto alla Versione 3

La Versione 3 si concentrava sul Parser. Il Parser è diventato generico e basato sui parametri.

La Versione 4 non modifica il Parser come concetto principale. Il Parser continua a leggere i parametri richiesti dallo strumento selezionato e a estrarre gli argomenti necessari.

Il nuovo componente è il Planner.

Il flusso cambia da:

Richiesta → Router → Generic Parser → Executor → Memoria

a:

Richiesta → Router → Planner → Generic Parser → Executor → Memoria

Il Router continua a selezionare lo strumento più adatto. Il Planner costruisce quindi un piccolo piano di esecuzione attorno allo strumento selezionato.

## Nuovo concetto: Planner

Il Planner è un nuovo componente che crea un piano prima che l'agente analizzi gli argomenti ed esegua uno strumento.

In questa versione, il Planner è semplice e basato su regole. Utilizza il Router esistente per scegliere lo strumento più adatto.

Se viene trovato uno strumento adatto, il Planner restituisce un piano con lo strumento selezionato, il motivo del suo utilizzo e i parametri richiesti.

Esempio:

```
{
  "tool": "addition",
  "reason": "Somma due numeri.",
  "needs_arguments": ["a", "b"]
}
```

Questo piano viene quindi memorizzato nello stato dell'agente.

## **Richieste non supportate**

Il Planner gestisce anche le richieste non supportate.

Per esempio, la richiesta:

```
agent("Che tempo fa a Roma?")
```

non corrisponde ad alcuno strumento disponibile.

In questo caso, il Planner restituisce:

```
{
  "tool": None,
  "reason": "Non è stato trovato uno strumento adatto alla richiesta.",
  "needs_arguments": []
}
```

L'Agent restituisce quindi un risultato alternativo sicuro:

```
"Non posso ancora gestire questa richiesta."
```

Questo è importante perché l'agente deve fallire in modo chiaro e sicuro quando una richiesta non rientra nelle sue capacità attuali.

## **Architettura completa nella Versione 4**

Il flusso completo della Versione 4 è:

```
Richiesta → Router → Planner → Generic Parser → Executor → Memoria
↓
Output
```

Le responsabilità principali ora sono più chiare:

```
Router = seleziona lo strumento
Planner = crea il piano di esecuzione
Parser = estrae gli argomenti

Executor = convalida ed esegue
Agent = coordina
Memory = memorizza lo stato
```

L'Agent è ancora il coordinatore. Ora coordina pianificazione, parsing, esecuzione e memorizzazione.

## Stato dell'Agent nella Versione 4

Lo stato dell'agente ora include una nuova chiave: plan.

Uno stato tipico appare così:

```
{
  "request": "Quanto fa 2 + 3?",
  "plan": {
    "tool": "addition",
    "reason": "Somma due numeri.",
    "needs_arguments": ["a", "b"]
  },
  "action": "addition",
  "arguments": {"a": 2, "b": 3},
  "result": 5
}
```

Questo rende più semplice ispezionare il comportamento interno dell'agente.

## Test

I test verificano che l'agente continui a funzionare dopo l'aggiunta del Planner.

I principali casi di test sono:

```
agent("Quanto fa 2 + 3?")
agent("Ciao, quanto fa 2 + 3?")
agent("Ciao, mi chiamo luca.")
agent("Che tempo fa a Roma?")
memory
```

Questi test verificano che:

- il Router continui a dare priorità all'intento principale;
- il Planner crei un piano prima dell'esecuzione;
- il Parser continui a estrarre gli argomenti corretti;
- l'Executor continui a convalidare ed eseguire lo strumento selezionato;
- le richieste non supportate vengano gestite senza selezionare uno strumento;
- la Memory memorizzi il piano insieme al resto dello stato.

## Cosa insegna la Versione 4

La Versione 4 insegna un importante principio di progettazione degli agenti:

un agente dovrebbe pianificare prima di agire

Il Planner è ancora semplice, ma introduce l'idea che un agente possa creare una rappresentazione intermedia prima dell'esecuzione.

Ciò rende l'architettura più facile da estendere in futuro, soprattutto quando il progetto passerà a planner più avanzati e ad agenti multi-step.

## Limitazioni

La Versione 4 migliora l'architettura, ma rimangono alcune limitazioni:

- il Router è ancora basato su regole;
- il Planner è ancora basato su regole;

- il Parser utilizza ancora regole di estrazione manuali;
- la memoria è ancora una semplice lista;
- non esiste ancora un Tool Registry;
- non è ancora presente alcun LLM;
- l'agente esegue ancora una sola azione alla volta.

Queste limitazioni sono previste e verranno affrontate gradualmente nelle versioni successive.

## **Conclusione**

La Versione 4 è un'evoluzione naturale della Versione 3.

Il Generic Parser rimane invariato, ma il nuovo elemento centrale è il Planner.

L'agente ora crea un piccolo piano di esecuzione prima di analizzare gli argomenti ed eseguire lo strumento selezionato.

Questo rende l'agente più trasparente e prepara il progetto a comportamenti più avanzati.

Il prossimo passo sarà la Versione 5, nella quale il progetto introdurrà un Memory Manager.